

Class 17: Version Control with Git

Data-driven Systems Engineering

Agenda

- Why version control matters for a growing modeling codebase, and soon, a team project.
- Git fundamentals: commits, history, and diffs.
- Branching and merging, including conflicts.
- Remotes: push, pull, and collaborating through GitHub.
- Pull requests and code review as a team practice.
- What never belongs in a commit.

Learning goals

- Initialize and use a Git repository for day-to-day modeling work.
- Write commits that form a usable, reviewable history.
- Branch to isolate risky or exploratory changes, and resolve a merge conflict without losing work.
- Collaborate through a shared remote using pull requests and code review.
- Recognize what belongs in '.gitignore' and what should never be committed at all.

Course Map and Running Cases



Today in the course

Focus: Getting code, notebooks, and a growing project under reliable, reviewable version control.

Bridge: Class 16 tracked which model and hyperparameters won. This class tracks the code and history behind every one of those runs, and builds the habits the team project (Classes 32-34) will require.

Fraud case

In the fraud case, version control tracks changes to the scoring pipeline, notebooks, and Streamlit app as they evolve across the course.

Pasteurization case

In the pasteurization case, it tracks changes to the synthetic sensor generator, streaming logic, and dashboard the same way.

Course thread: The course thread moves from individually tracked experiments to a shared, reviewable

Why version control, and why now

- Every prior class produced code that changed: notebooks, a Streamlit app, feature pipelines.
- Without version control, “which version produced this result” is a guess, not an answer.
- Git gives every experiment, threshold choice, and interface change a traceable history.
- This is a practical habit to build now, before project work in Classes 32-34 makes it unavoidable.

Git fundamentals

```
git init
git clone <url>
git status
git add <file>
git commit -m "message"
git log --oneline
git diff
```

- A commit is a snapshot with a message, an author, and a parent – together they form a history, not just a backup.
- 'git diff' and 'git log' turn “what changed, and why” into an answerable question instead of a guess.

What makes a good commit

- Small and atomic: one logical change per commit, not a day's worth of edits at once.
- A message that explains why, not just what – the diff already shows what changed.
- Committed often enough that 'git log' reads like a story, not a single “final version” checkpoint.
- Working code at each commit, where practical, so history stays useful for debugging and rollback.

Branching to try an idea safely

```
git checkout -b experiment/cost-sensitive-threshold
git add .
git commit -m "Try cost-sensitive threshold tuning"
git push -u origin experiment/cost-sensitive-threshold
# compare results, then merge or discard the branch
```

- A branch isolates a risky or exploratory change so 'main' keeps the last known-good model and interface.
- If the experiment does not beat the baseline, the branch can simply be discarded – no cleanup, no regret.

Merging, and when it goes wrong

```
git checkout main
git merge experiment/cost-sensitive-threshold
# CONFLICT (content): Merge conflict in scoring.py
```

- Git marks the conflicting region with '«««<', '=====', and '»»»>' – it needs a human decision, not a tool trick.
- Resolving a conflict means choosing (or combining) the intended behavior, then committing the resolution.
- Merge conflicts are expected, not exceptional; resolving them is part of collaborative engineering, not a sign of failure.

Remotes and collaborating through GitHub

```
git remote -v  
git push origin main  
git pull origin main  
git fetch
```

- A remote is a shared copy of the repository – GitHub, in this course's case.
- 'push' sends local commits to the remote; 'pull' fetches and merges the remote's new commits into the local branch.
- Pulling before starting new work avoids diverging unnecessarily from teammates.

Pull requests and code review

- A pull request is a review checkpoint before code reaches a shared branch – exactly where a teammate can catch a silent behavior change.
- Small pull requests are easier to review honestly than large ones.
- Review comments are about the code and the decision behind it, not the author.
- For the practical project, agree on a branching convention (for example, one feature branch per teammate or per task) before the first commit, not after the first conflict.

What never belongs in a commit

```
/.ipynb_checkpoints  
/venv  
.DS_Store  
  
# Python caches  
__pycache__/  
*.py[cod]
```

- This is this repository's own '.gitignore': build artifacts, virtual environments, and editor files never need a history.
- Secrets, API keys, and credentials never belong in a commit – not even one that gets “fixed” later, since the history remains.
- Large trained artifacts are a judgment call: sometimes worth committing, sometimes better handled outside Git entirely.

Repo activity

- This repository's own commit history and '.gitignore' are a live example of the ideas in this class.
- 'notebooks/', 'streamlit/', and 'pasteurization/' are exactly the kind of fast-changing code that benefits from small, frequent commits.
- The practical project (Classes 32-34) is where each team applies this to its own shared repository.

Question

Two teammates each push a working fix for the same bug, on different branches, without talking first. What went wrong before the merge conflict even happened?

- This separates “we can merge this” from “we should have coordinated before writing it.”
- It motivates lightweight team conventions (issue assignment, small pull requests, short-lived branches) over relying on Git to resolve coordination problems after the fact.

Papers and materials

Pro Git (free online book); GitHub documentation on pull requests; Driessen, A Successful Git Branching Model; Atlassian guide to resolving merge conflicts.

From This Class to the Next

What stays with us

- Version control turns “which version produced this” from a guess into an answer.
- Branches isolate risk; pull requests turn merging into a review checkpoint, not a surprise.
- What never gets committed (secrets, build artifacts) matters as much as what does.

Project use

Teams should set up a shared repository with an agreed branching convention before their first project commit – not after the first conflict.

Next connection

Class 18 moves from a fixed, versioned snapshot to a model that must keep working as new data keeps arriving: online learning and concept drift.

Course thread: A reviewable history is what makes every later engineering practice in this course – CI/CD, rollback, monitoring – possible in the first place.